FINAL VIEW OF HELM 3–4 YOE interview format

1. Helm chart install is failing. What will you check?
Answer:
First, I check the Helm error to identify whether the issue is with the chart, values, or Kubernetes.
Then I use helm lint and helm template to check template and YAML issues.
If rendering is fine, I check namespace, RBAC, CRDs, resource conflicts, and Kubernetes events.
After identifying the root cause, I fix it and retry the installation.
Simple ga gurthupettuko:
Error → Lint/Template → K8s/RBAC/CRD → Events → Fix & Retry
Commands:
helm lint ./mychart
helm template myapp ./mychart -f values.yaml
helm status myapp -n dev
kubectl get events -n dev --sort-by=.lastTimestamp
kubectl auth can-i create deployments -n dev

---

2. Helm upgrade is failing. What will you check?
Answer:
First, I check the upgrade error and identify which resource or configuration is causing the failure.
Then I compare the current values and rendered manifest with the previous working release.
I check Kubernetes events, resource configuration, image, probes, and any dependency or permission issues.
After finding the root cause, I fix it and retry the upgrade. If required, I rollback to the last known-good revision.
Simple ga gurthupettuko:
Error → Old vs New → Events/Resources → Fix → Rollback if needed
Commands:
helm status myapp -n dev
helm history myapp -n dev
helm get values myapp -n dev
kubectl get events -n dev --sort-by=.lastTimestamp
helm rollback myapp <revision> -n dev

---

3. Helm upgrade succeeded but application is not working. What will you check?
Answer:
First, I verify the Helm release and Deployment status.
Then I check Pod readiness, logs, events, Service, ports, and the deployed image.
I also compare the current release with the previous working revision to identify what changed.
Based on the root cause, I fix the configuration or rollback if required.
Simple ga gurthupettuko:
Upgrade success → Pods → Logs → Service → Compare old release → Fix/Rollback
Commands:
helm status myapp -n dev
kubectl get pods -n dev
kubectl logs <pod_name> -n dev
kubectl get svc myapp -n dev
helm history myapp -n dev

---

4. Helm works in DEV but fails in PROD. What will you check?
Answer:
First, I compare the Helm chart version and values used in DEV and PROD.
Then I check environment-specific Secrets, ConfigMaps, image tags, resources, namespaces, RBAC, and cluster differences.
I identify the exact difference between the environments and fix that configuration.
After validation, I redeploy to PROD.
Simple ga gurthupettuko:
DEV works → DEV vs PROD compare → Config/Secret/RBAC/Resources → Difference fix
Commands:
helm get values myapp -n dev
helm get values myapp -n prod
helm list -n prod
kubectl get secret -n prod
kubectl get configmap -n prod

---

5. Helm release is stuck in pending state. What will you check?
Answer:
First, I check the Helm status and history to understand which operation is pending.
Then I check Kubernetes events, Pods, Jobs, and Helm hooks related to that release.
I also verify whether the previous operation failed or another deployment process is still running.
Once I identify the cause, I recover the release or rollback to a known-good revision.
Simple ga gurthupettuko:
Pending → Status/History → Pods/Jobs/Hooks → Root cause → Recover/Rollback

Commands:
helm status myapp -n dev
helm history myapp -n dev
kubectl get pods -n dev
kubectl get jobs -n dev
kubectl get events -n dev --sort-by=.lastTimestamp

---

6. How do you rollback a failed Helm deployment?
Answer:
First, I check the Helm history and identify the last known-good revision.
Then I rollback the release to that revision.
After rollback, I verify the Deployment, Pods, application health, and Helm status.
Finally, I investigate the failed release separately so the same issue doesn't happen again.
Simple ga gurthupettuko:
History → Last good revision → Rollback → Verify → Root cause
Commands:
helm history myapp -n prod
helm rollback myapp <revision> -n prod
helm status myapp -n prod
kubectl get pods -n prod
kubectl rollout status deployment/myapp -n prod

---

7. Helm values are not being applied. What will you check?
Answer:
First, I verify that I am using the correct values.yaml file.
Then I check the value key, YAML structure, multiple values files, and --set overrides.
I use helm template to verify what value is actually being rendered into the Kubernetes manifest.
If the rendered value is correct but the application still doesn't reflect it, I check the running resource and Pod configuration.
Simple ga gurthupettuko:
Values file → Key → Override → Render → Running resource
Commands:
cat values.yaml
helm get values myapp -n dev
helm template myapp ./mychart -f values.yaml
helm get manifest myapp -n dev
kubectl get deployment myapp -n dev -o yaml

---

8. Wrong value is getting picked from values.yaml. What will you check?
Answer:
First, I check all the values files passed to Helm because later files can override earlier values.
Then I check --set parameters and the exact key path used in the template.
I render the chart and verify the final value.
Once I find the override or key mismatch, I correct it and upgrade the release.
Simple ga gurthupettuko:
Values files → --set **→ Template key → Render → Fix**
Commands:
helm upgrade myapp ./mychart -f values.yaml -f values-prod.yaml -n prod
helm template myapp ./mychart -f values.yaml
grep -R "image.tag" ./mychart/templates
helm get values myapp -n prod

---

9. How do you manage DEV, QA, UAT, and PROD using Helm?
Answer:
I maintain a common Helm chart and keep environment-specific configuration separately.
For example, I use separate values files for replicas, resources, image tags, endpoints, and environment configuration.
The deployment logic remains common, while only environment-specific values change.
This gives consistency and avoids maintaining separate charts for every environment.
Simple ga gurthupettuko:
One chart → Environment values → Same logic → Controlled deployment
Commands:
helm upgrade --install myapp ./mychart -f values-dev.yaml -n dev
helm upgrade --install myapp ./mychart -f values-qa.yaml -n qa
helm upgrade --install myapp ./mychart -f values-uat.yaml -n uat
helm upgrade --install myapp ./mychart -f values-prod.yaml -n prod

---

10. Helm template rendering is failing. What will you check?
Answer:
First, I check the exact template error.
Then I validate the chart using helm lint and render it using helm template.
I check missing values, incorrect template expressions, YAML indentation, and wrong references.
After fixing the template issue, I render it again to verify the generated manifest.

Simple ga gurthupettuko:
Error → Lint → Template → Values/Syntax → Render again
Commands:
helm lint ./mychart
helm template myapp ./mychart -f values.yaml
cat values.yaml
helm show values ./mychart

---

11. Helm template renders successfully but Kubernetes deployment fails. What will you check?
Answer:
If helm template succeeds, I know the basic Helm rendering is working.
Then I check the Kubernetes API error, resource configuration, image, probes, permissions, CRDs, and namespace.
I also check Pod events to identify the actual runtime problem.
After fixing the Kubernetes-side issue, I run the Helm upgrade again.
Simple ga gurthupettuko:
Template OK → K8s error → Resource/Permission/CRD → Events → Fix
Commands:
helm template myapp ./mychart -f values.yaml
helm status myapp -n dev
kubectl get events -n dev --sort-by=.lastTimestamp
kubectl describe pod <pod_name> -n dev
kubectl auth can-i create deployments -n dev

---

12. Wrong Docker image tag is deployed through Helm. What will you check?
Answer:
First, I check the image repository and tag in the Helm values.
Then I check whether another values file or --set is overriding the expected tag.
I verify the rendered Deployment and the image configured on the running Pods.
If the tag is wrong, I correct it and perform a Helm upgrade.
Simple ga gurthupettuko:
Values → Override → Rendered image → Pod image → Upgrade
Commands:
helm get values myapp -n dev
helm template myapp ./mychart -f values.yaml | grep "image:"
kubectl get deployment myapp -n dev -o jsonpath='{.spec.template.spec.containers[*].image}'
kubectl get pods -n dev -o jsonpath='{.items[*].spec.containers[*].image}'

---

13. Helm upgrade is deploying the old Docker image. What will you check?
Answer:
First, I verify the image tag passed during the Helm upgrade.
Then I check the rendered Deployment and confirm which image Kubernetes is actually using.
I also check whether the registry contains the expected image and whether the Pods were recreated with the new version.
I prefer unique immutable image tags instead of relying on latest.
Simple ga gurthupettuko:
Tag → Render → Registry → Pod image → Unique tag
Commands:
helm get values myapp -n dev
helm get manifest myapp -n dev
kubectl get deployment myapp -n dev -o jsonpath='{.spec.template.spec.containers[*].image}'
kubectl get pods -n dev -o wide

---

14. Helm chart dependency is failing. What will you check?
Answer:
First, I check the dependency definition and version in Chart.yaml.
Then I verify whether the required Helm repository is configured and the requested chart version exists.
I also check dependency download errors and repository connectivity.
After fixing the version or repository configuration, I update the dependencies and retry the deployment.
Simple ga gurthupettuko:
Chart.yaml → Version → Repo → Dependency update → Retry
Commands:
cat Chart.yaml
helm repo list
helm repo update
helm dependency update ./mychart
helm dependency list ./mychart

---

15. Helm repository is unavailable. What will you check?
Answer:
First, I verify the repository URL and Helm repository configuration.
Then I check DNS, network connectivity, proxy settings, and authentication if required.
I also verify whether the repository itself is available.

Once connectivity is restored, I update the repository and retry the Helm operation.
Simple ga gurthupettuko:
URL → DNS/Network → Proxy/Auth → Repo health → Retry
Commands:
helm repo list
helm repo update
nslookup <repo-domain>
curl -I <repo-url>

---

16. Helm release shows deployed, but the application is not accessible. What will you check?
Answer:
First, I verify that the Pods are actually Running and Ready.
Then I check the Service selector, ports, targetPort, Endpoints, and Ingress configuration.
I also check readiness probes and application logs.
This helps me identify whether the issue is inside the Pod or in the traffic path.
Simple ga gurthupettuko:
Pods → Service → Endpoints → Ingress → Logs/Probes
Commands:
helm status myapp -n prod
kubectl get pods -n prod
kubectl get svc myapp -n prod
kubectl get endpoints myapp -n prod
kubectl get ingress -n prod

---

17. Helm upgrade causes Pods to restart unexpectedly. What will you check?
Answer:
First, I compare the old and new release configuration.
Then I check changes to the image, environment variables, ConfigMaps, Secrets, probes, and resource settings.
I also check Deployment rollout history and Pod events to understand why the restart happened.
If the change is causing instability, I fix it or rollback to the previous stable revision.
Simple ga gurthupettuko:
Old vs New → Image/Config/Probe → Events → Fix/Rollback
Commands:
helm history myapp -n prod
helm get manifest myapp -n prod
kubectl rollout history deployment/myapp -n prod
kubectl get events -n prod --sort-by=.lastTimestamp
kubectl get pods -n prod

---

18. Helm deployment is partially successful. What will you check?
Answer:
First, I identify which resources were successfully created and which ones failed.
Then I check Helm status, Kubernetes events, failed Pods or Jobs, and the release history.
I identify whether the failure is related to configuration, image, resources, permissions, or dependencies.
After fixing the root cause, I complete the deployment or rollback if necessary.
Simple ga gurthupettuko:
Success/Fail resources → Events → Root cause → Fix → Complete/Rollback
Commands:
helm status myapp -n dev
kubectl get all -n dev
kubectl get events -n dev --sort-by=.lastTimestamp
kubectl get pods -n dev
helm history myapp -n dev

---

19. Helm hook is causing deployment failure. What will you check?
Answer:
First, I identify which Helm hook is failing.
Then I check the hook Job or Pod logs, events, image, permissions, and dependencies.
I verify whether the hook is blocking the Helm operation.
After fixing the hook issue, I rerun the deployment and verify the release.
Simple ga gurthupettuko:
Hook → Job/Pod → Logs/Events → Permissions → Fix
Commands:
helm status myapp -n dev
kubectl get jobs -n dev
kubectl get pods -n dev
kubectl logs job/<job_name> -n dev
kubectl describe job <job_name> -n dev

20. Two Helm releases are conflicting with the same Kubernetes resource. What will you check?
Answer:
First, I identify the resource both releases are trying to manage.
Then I check the namespaces, release manifests, and Helm ownership metadata.
I verify whether the resource is hardcoded or intentionally shared between the charts.
I then correct the chart design so that only one release manages that resource.
Simple ga gurthupettuko:
Common resource → Ownership → Manifest → Chart fix
Commands:
helm list -A
helm get manifest myapp -n dev
helm get manifest myapp2 -n dev
kubectl get deployment -A
kubectl get deployment <name> -n dev -o yaml

---

21. Helm release was accidentally deleted. What will you do?
Answer:
First, I assess the impact and check whether the Kubernetes resources are still running.
Then I identify the previous chart version and configuration from available release information or backups.
If the application is affected, I restore the release using the correct chart version and values.
After recovery, I verify application health and investigate how the deletion happened.
Simple ga gurthupettuko:
Impact → Previous config → Restore → Verify → Root cause
Commands:
helm list -A
kubectl get all -n prod
helm history myapp -n prod
helm install myapp ./mychart -n prod -f values-prod.yaml
kubectl get pods -n prod

---

22. How do you safely upgrade a Production Helm release?
Answer:
First, I test the same chart and configuration in a lower environment.
Then I review the rendered manifests, image version, configuration, and rollback revision.
For Production, I perform a controlled upgrade and monitor the rollout and application health.
If anything goes wrong, I rollback to the last known-good revision.
Simple ga gurthupettuko:
Test → Validate → Backup revision → Upgrade → Monitor → Rollback
Commands:
helm lint ./mychart
helm template myapp ./mychart -f values-prod.yaml
helm history myapp -n prod
helm upgrade myapp ./mychart -n prod -f values-prod.yaml
kubectl rollout status deployment/myapp -n prod

---

23. How do you verify a Helm deployment before Production rollout?
Answer:
First, I validate the chart and render the final Production manifest.
Then I verify image, resources, probes, Services, configuration, and environment-specific values.
I test the same chart version in a lower environment.
Only after successful validation do I promote it to Production.
Simple ga gurthupettuko:
Lint → Render → Config/Image check → Lower env → Prod
Commands:
helm lint ./mychart
helm template myapp ./mychart -f values-prod.yaml
helm upgrade myapp ./mychart -n qa -f values-qa.yaml
kubectl rollout status deployment/myapp -n qa

---

24. How do you troubleshoot Helm + Kubernetes deployment failure?
Answer:
First, I check the Helm error and release status.
Then I render the chart and verify whether the problem is coming from Helm values or Kubernetes.
If the manifest is correct, I check Kubernetes events, Pod status, logs, permissions, and resource configuration.
Once I identify the root cause, I fix the chart or configuration and perform a controlled upgrade.
Simple ga gurthupettuko:
Helm error → Render → K8s events/logs → Root cause → Fix/Upgrade
Commands:
helm status myapp -n dev
helm template myapp ./mychart -f values.yaml
kubectl get events -n dev --sort-by=.lastTimestamp

kubectl get pods -n dev
kubectl logs <pod_name> -n dev

25. How do you manage Helm chart versions in Production?

Answer:

I track the Helm chart version separately from the application image version.
Before upgrading Production, I identify the current chart version and test the target version in a lower environment.
I keep the previous known-good revision available for rollback.
This gives controlled upgrades and a clear recovery path.

Simple ga gurthupettuko:

Current version → Test new version → Prod upgrade → Keep rollback path

Commands:

helm list -n prod
helm show chart ./mychart
helm history myapp -n prod
helm upgrade myapp ./mychart -n prod -f values-prod.yaml

---

26. Helm deployment is using unexpected replicas or resources. What will you check?

Answer:

First, I check the configured replica count, CPU, and memory values.
Then I check whether another values file or --set is overriding them.
I render the chart and verify the actual values in the Deployment manifest.
If the values are wrong, I correct them and perform the Helm upgrade.

Simple ga gurthupettuko:

Values → Override → Rendered Deployment → Fix/Upgrade

Commands:

helm get values myapp -n dev
helm template myapp ./mychart -f values.yaml
kubectl get deployment myapp -n dev -o yaml
kubectl describe deployment myapp -n dev

---

27. Helm chart works on one Kubernetes cluster but fails on another. What will you check?

Answer:

First, I compare the Kubernetes and Helm versions of both clusters.
Then I check API versions, CRDs, RBAC permissions, admission policies, and available resources.
I also compare the chart version and values used in both clusters.
Once I identify the cluster-specific difference, I fix the chart or environment configuration.

Simple ga gurthupettuko:

Versions → APIs/CRDs → RBAC → Resources → Difference fix

Commands:

kubectl version
helm version
kubectl api-resources
kubectl get crd
kubectl auth can-i create deployments -n prod

---

28. Helm Secret/Config changes are not reflected after upgrade. What will you check?

Answer:

First, I verify that the updated Secret or ConfigMap is actually rendered and updated in Kubernetes.
Then I check how the application consumes it — through environment variables or mounted files.
If the values are injected as environment variables, existing Pods usually need to be recreated to pick up the new values.
I restart the workload if required and verify the updated configuration.

Simple ga gurthupettuko:

Render → Secret/ConfigMap → How Pod consumes → Restart if needed → Verify

Commands:

helm get manifest myapp -n dev
kubectl get configmap <name> -n dev -o yaml
kubectl get secret <name> -n dev
kubectl rollout restart deployment/myapp -n dev
kubectl rollout status deployment/myapp -n dev

---

29. How do you prevent accidental Production Helm deployments?

Answer:

I restrict Production Kubernetes and Helm access using RBAC.
I keep Production values and credentials separate from lower environments.
In CI/CD, I use protected branches, approval gates, and controlled deployment permissions.
This ensures that only authorized and approved changes reach Production.

Simple ga gurthupettuko:

RBAC → Separate Prod config → Approval → Controlled deployment

Commands:

kubectl auth can-i create deployments -n prod
helm list -n prod

helm upgrade myapp ./mychart -n prod -f values-prod.yaml --dry-run

30. Explain a real-time CI/CD → Helm → Kubernetes deployment scenario.

Answer:

Developer pushes code to Git, which triggers the CI pipeline.

CI builds and tests the application, creates a versioned Docker image, and pushes it to the registry.

Then the CD pipeline deploys the new image using Helm with the appropriate environment values.

After deployment, I verify the rollout, Pod health, application health, and monitoring. If there is a Production issue, I rollback to the previous known-good Helm revision.

Simple ga gurthupettuko:

Git → CI → Docker → Registry → Helm → Kubernetes → Health Check → Rollback

Commands:

git push origin main

docker build -t myapp:v1.2.0 .

docker push registry.example.com/myapp:v1.2.0

helm upgrade myapp ./mychart -n prod --set image.tag=v1.2.0

kubectl rollout status deployment/myapp -n prod

helm rollback myapp <revision> -n prod

Ivi final ga mana target format: short answer + practical depth + commands + memory line.